

September 8, 2026

rNat Patch

Smart Contract Patch Review

Placeholder text consisting of a repeating pattern of stylized symbols.

Contents	About Zellic	3
1.	Introduction	3
1.1.	Scope	4
1.2.	Disclaimer	5
2.	Detailed Findings	5
2.1.	Inconsistent reentrancy protection across RNat account operations	6
3.	Patch Review	6
3.1.	RNatAccount creation-time reentrancy issue fix	7

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Introduction

We were asked to review a patch to the RNat subsystem from September 3rd to September 4th, 2026 for a total of 0.6 person-weeks.

1.1. Scope

The engagement involved a review of the following targets:

rNat Patch Contracts

Type	Solidity
Platform	EVM-compatible
Target	Only changes between b69873e...ecdeafc
Repository	https://github.com/flare-foundation/flare-smart-contracts-v2 ↗
Version	ecdeafcb4f4ed14eac214e19c3b1000a48661f72
Programs	contracts/rNat/implementation/RNatAccount.sol

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✧ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Weipeng Lai
✧ Engineer
weipeng.lai@zellic.io ↗

Ayaz Mammadov
✧ Engineer
ayaz@zellic.io ↗

1.2. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.

2. Detailed Findings

2.1. Inconsistent reentrancy protection across RNat account operations

Target	RNat		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

RNat applies `nonReentrant` to `claimRewards` and `transferExternalToken`, but not to `setClaimExecutors`, `withdraw`, or `withdrawAll`. These unguarded functions can trigger callbacks to the account owner when returning executor-fee refunds or withdrawn native tokens. The owner can use these callbacks to reenter RNat because the guard does not cover all five operations.

RNat's `mustBalance` modifier burns native value that exceeds its expected balance, so its accounting must remain consistent whenever a callback enters a balance-checked function. Inconsistent guard coverage makes this depend on each operation's accounting and callback order. Applying a shared guard across these operations would make that protection more explicit.

Impact

This is a hardening recommendation. Consistent reentrancy protection would reduce the risk of future changes introducing callbacks that disrupt balance accounting.

Recommendations

Apply `nonReentrant` to the three unguarded user entry points: `setClaimExecutors`, `withdraw`, and `withdrawAll`.

The remaining `mustBalance` functions do not warrant the same protection because they are restricted by access control.

Remediation

This issue has been acknowledged by Flare.

3. Patch Review

This section documents the core changes in the codebase made within the in-scope commit range [b69873e1](#) to [ecdeafcb](#).

3.1. RNatAccount creation-time reentrancy issue fix

The commit message for [e9d6c94c](#) describes the issue as follows:

A pre-funded (CREATE-deterministic) RNatAccount address let the creation-time bootstrap `setClaimExecutors` hand control to the owner via `_transferCurrentBalanceToOwner` while RNat still held an unaccounted `msg.value` from the outer `setClaimExecutors` call. Re-entering a `mustBalance` function then burned that value as if it were self-destruct proceeds, permanently under-balancing RNat and bricking every `mustBalance` flow.

`RNat.setClaimExecutors` accepts native tokens and forwards them to the caller's `RNatAccount`, creating the account first if necessary. During creation, the incoming `msg.value` remains in RNat without being included in its expected balance. The function has neither `mustBalance` nor `nonReentrant` protection. An attacker could exploit this temporary balance discrepancy by pre-funding the next account's predictable CREATE address.

When the owner's executor fee is zero, account creation bootstraps the account through `RNatAccount.setClaimExecutors`, which ends by transferring any native balance to the owner through `_transferCurrentBalanceToOwner`. The pre-funded balance therefore triggers an attacker-controlled callback before RNat forwards the outer call's `msg.value`. During this callback, the attacker can re-enter `claimRewards` with an empty project list. Its `mustBalance` modifier treats the outer call's unaccounted value as surplus and burns it. If RNat has sufficient remaining funds to complete the outer transfer, the transaction succeeds but leaves its native balance below the expected balance. Subsequent calls to functions protected by `mustBalance` revert while this deficit persists.

The patch resolves the issue by adding a balance-neutralizing step to `RNatAccount.initialize`, which runs before the bootstrap `setClaimExecutors` call:

```
function initialize(  
    address _owner,  
    IRNat _rNat  
)  
    external  
{  
    require(address(owner) == address(0), "owner already set");  
    require(address(_owner) != address(0), "owner address zero");  
    require(address(_rNat) != address(0), "rNat address zero");  
    owner = _owner;  
    rNat = _rNat;  
    // Neutralize any pre-funding of this CREATE-deterministic address by  
    wrapping it into
```

```
// WNat. This ensures the creation-time bootstrap `setClaimExecutors`  
call finds a zero  
  
// native balance and cannot hand control to the owner (via `_transferCurr  
entBalanceToOwner`)  
// while `RNat` still holds an unaccounted `msg.value` from an outer `  
setClaimExecutors` call.  
uint256 preFunded = address(this).balance;  
if (preFunded > 0) {  
    // Destination is the canonical WNat from the trusted RNat; the  
    value is this  
    // account's own pre-fund. Safe despite the static-analysis  
    arbitrary-send flag.  
    //slither-disable-next-line arbitrary-send-eth  
    _rNat.wNat().deposit{value: preFunded}();  
}  
emit Initialized(owner, _rNat);  
}
```

This wraps the account's pre-existing native balance into WNat. Consequently, the bootstrap sweep finds no native balance and skips the owner callback, removing the reentrancy opportunity while the outer call's value remains in RNat.

The wrapped funds remain available to the owner as non-RNat funds. Wrapping increases the account's WNat balance without increasing `receivedRewards` or its monthly reward allocations. The pre-funded amount is therefore excluded from vesting, is withdrawn before RNat rewards, and does not affect RNat balances or total supply.